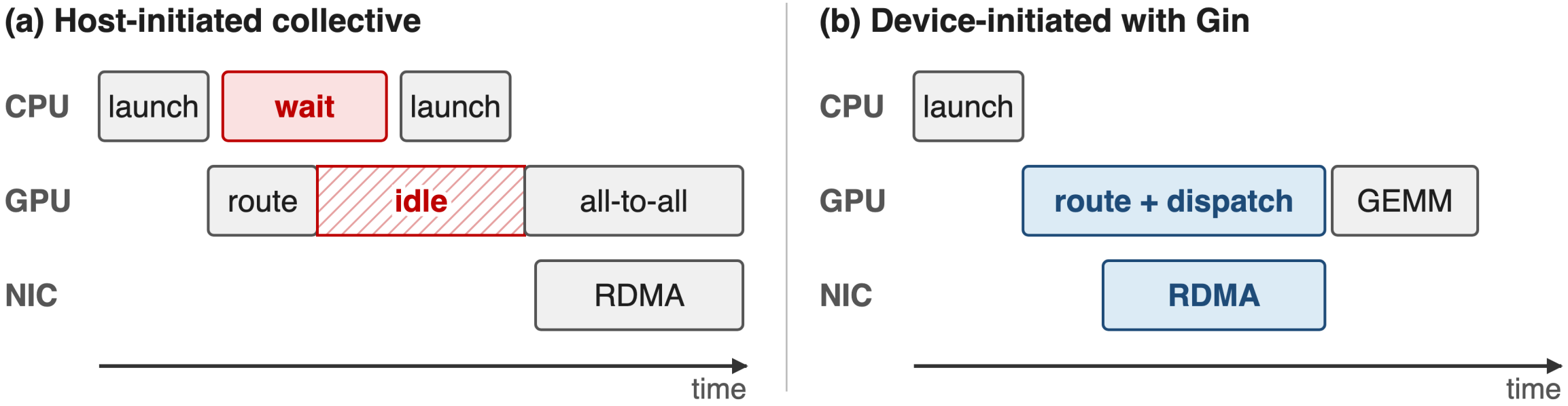
NCCL Gin backend

GPU-initiated RDMA inside NCCL, as DeepEP v2's network transport

DeepSeek-AI · DeepEP V2 / V2.5 on NVIDIA NCCL Device API (GIN)

Explainer deck · Industrial Pretraining Architecture & Infra Tech Landscape (2026)

Token routes are known only inside the kernel



Schematic, not to scale

Device-initiated RDMA lets the routing kernel post its own network writes

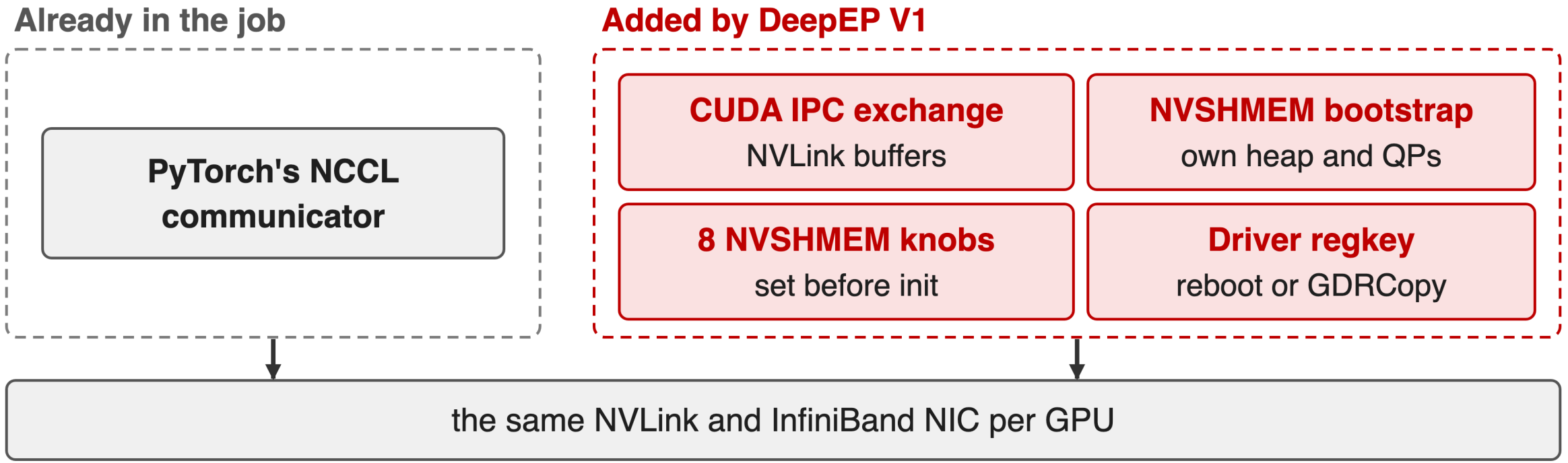
V1 needed 8 NVSHMEM knobs before one byte

8

NVSHMEM environment variables DeepEP V1 set before init, plus 3 all-gather handshakes and a driver regkey or GDRCopy

deep_ep/buffer.py at v1.2.1 (lines 85-115); third-party/README.md at v1.2.1

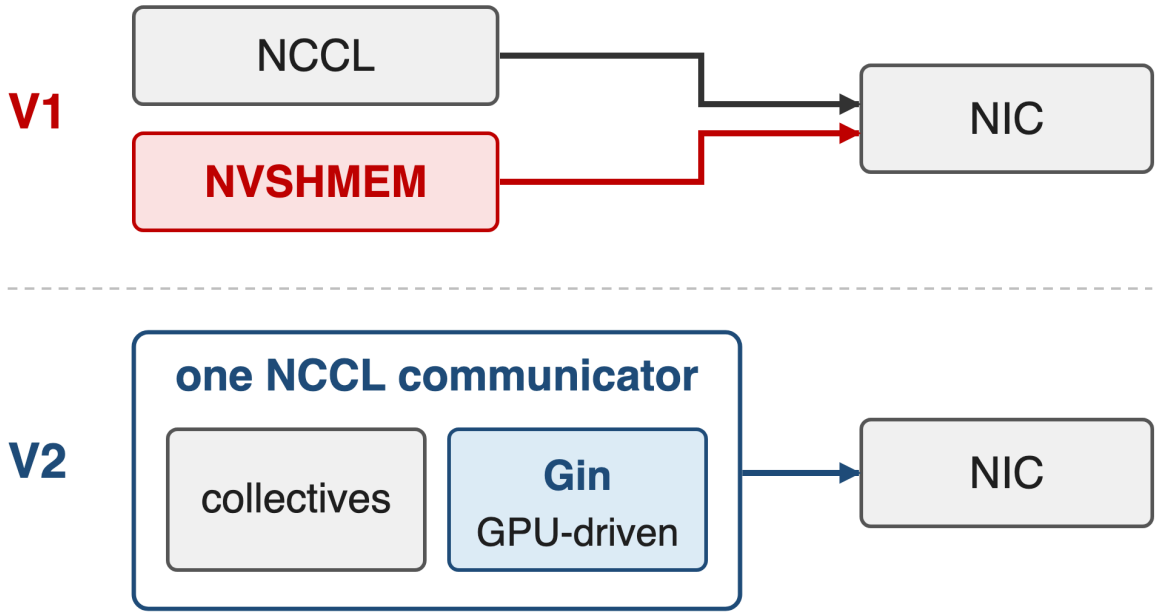
V1 ran a second runtime beside NCCL



Two bootstraps, two memory registrations, one NIC

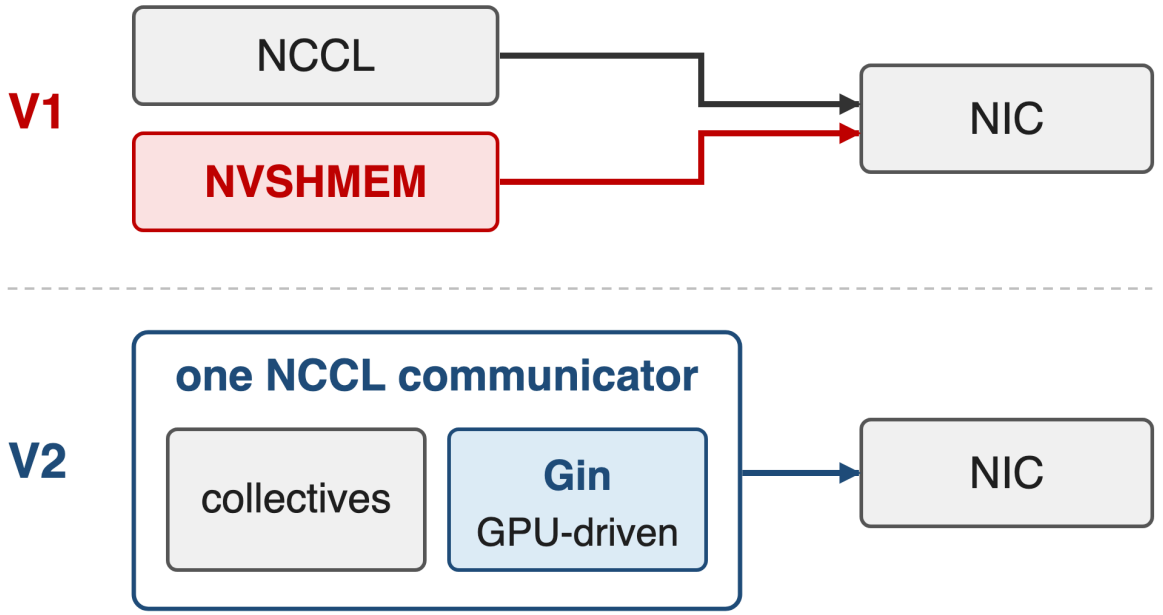
Insight: NCCL can hand GPUs the NIC doorbell

- A NIC needs only a work-queue entry and a doorbell write
- NCCL 2.28's Device API lets GPU threads do both
- One runtime covers collectives and GPU-driven RDMA



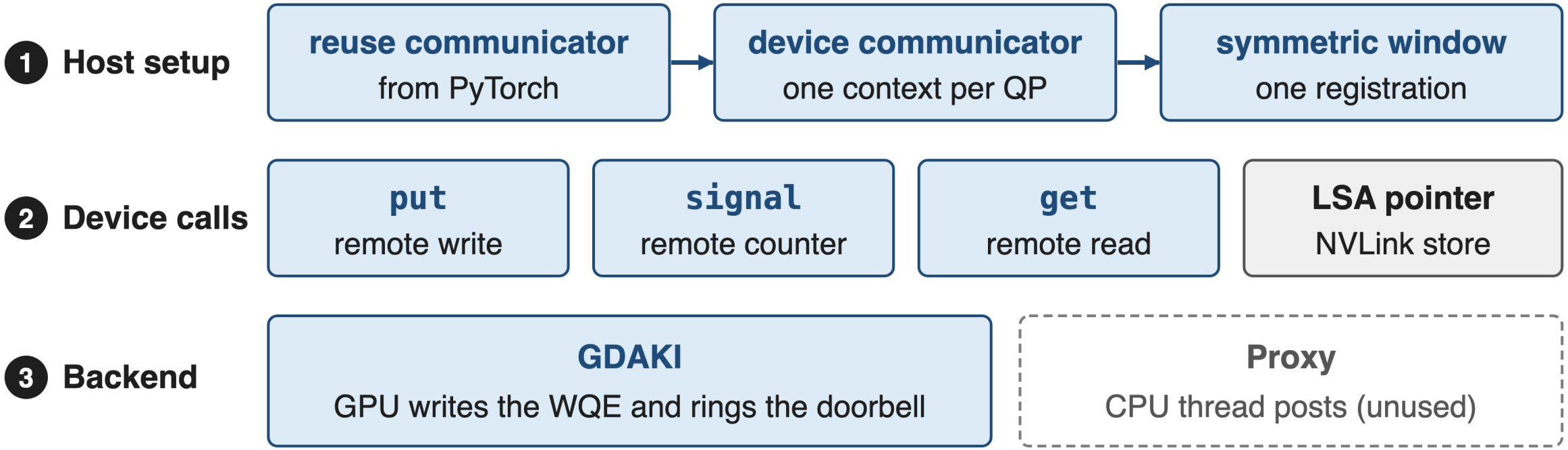
Insight: NCCL can hand GPUs the NIC doorbell

- A NIC needs only a work-queue entry and a doorbell write
- NCCL 2.28's Device API lets GPU threads do both
- One runtime covers collectives and GPU-driven RDMA



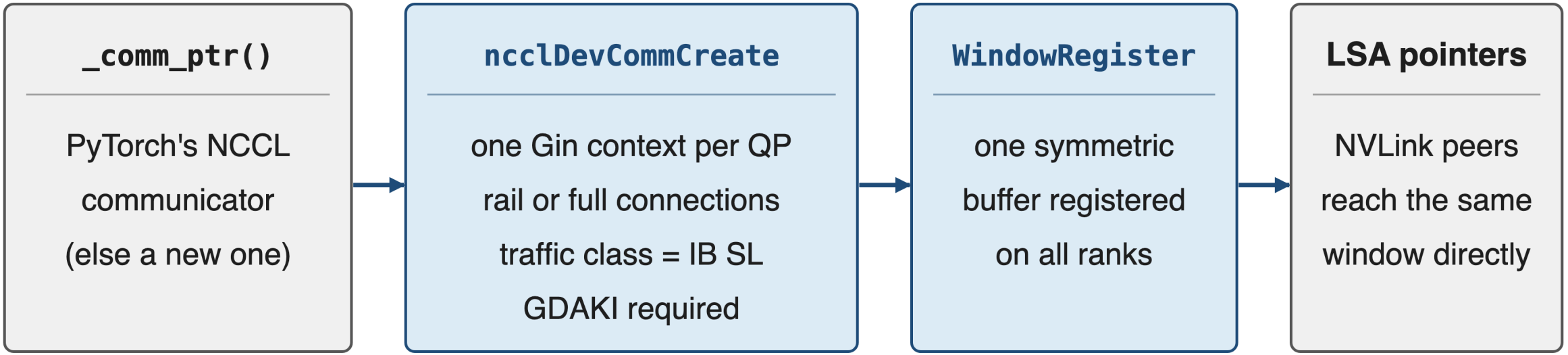
DeepEP v2: reuse PyTorch's communicator and drop NVSHMEM

Gin has three layers: setup, calls, backend



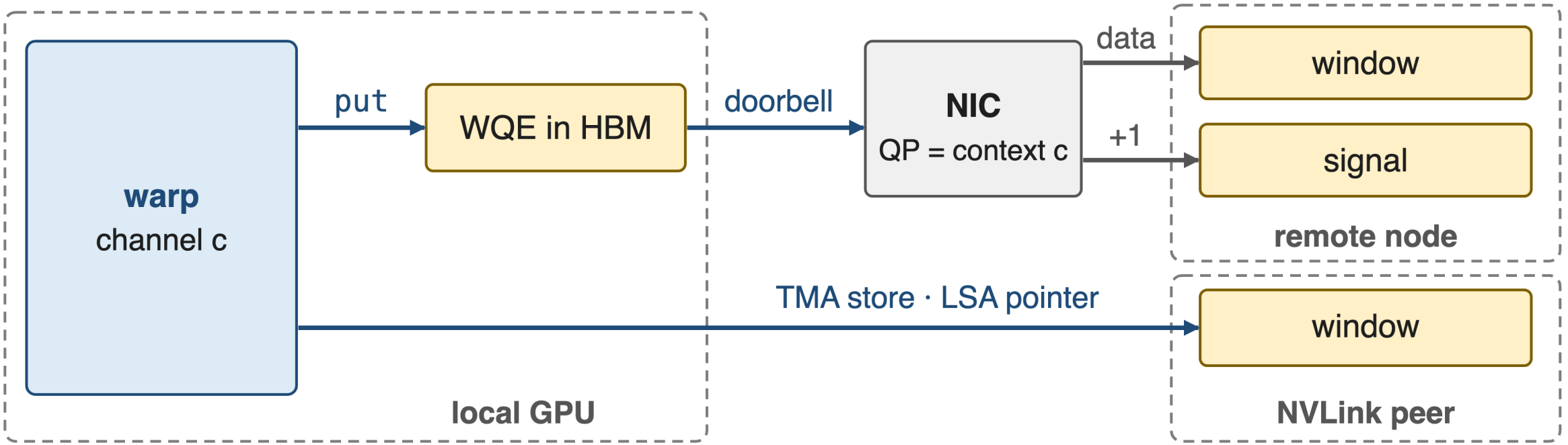
❶ host setup once per buffer · ❷ device calls inside DeepJIT-compiled kernels · ❸ GDAKI on the NIC

❶ Reuse one communicator and one window



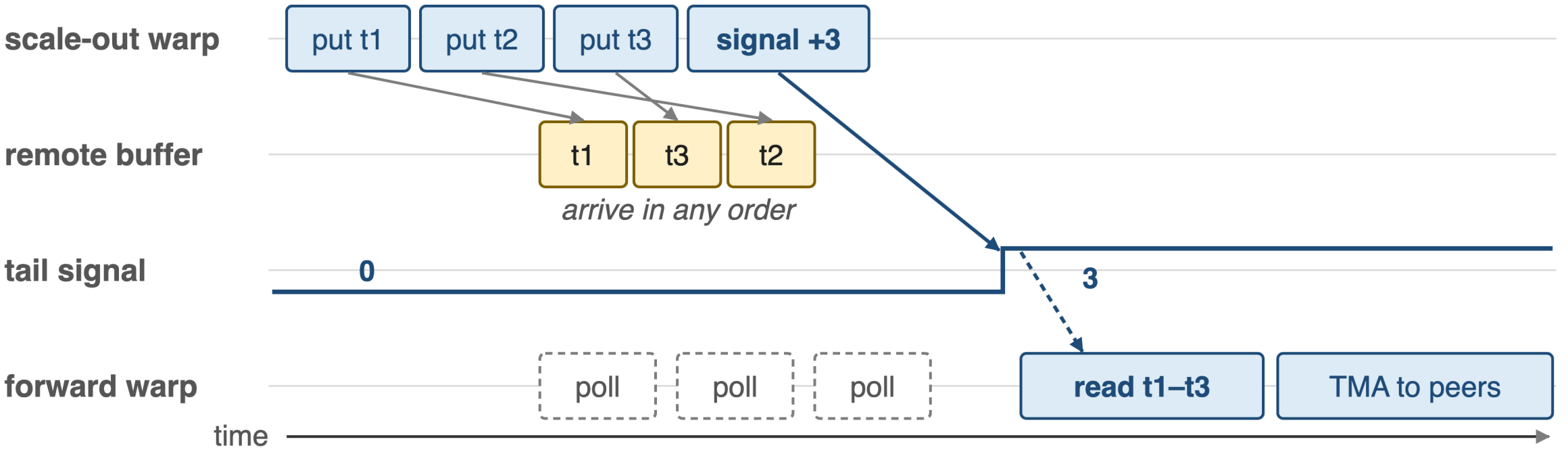
NCCL 2.28 offered 4 contexts per communicator, so the v1.2.1 port needed ⌈QPs/4⌉ communicators; V2 asks for one per QP

❷ A warp's put becomes a WQE and a doorbell



Remote nodes: put plus signal through the channel's QP. Same node: plain TMA stores through LSA pointers

❷ A signal after the puts replaces a fence



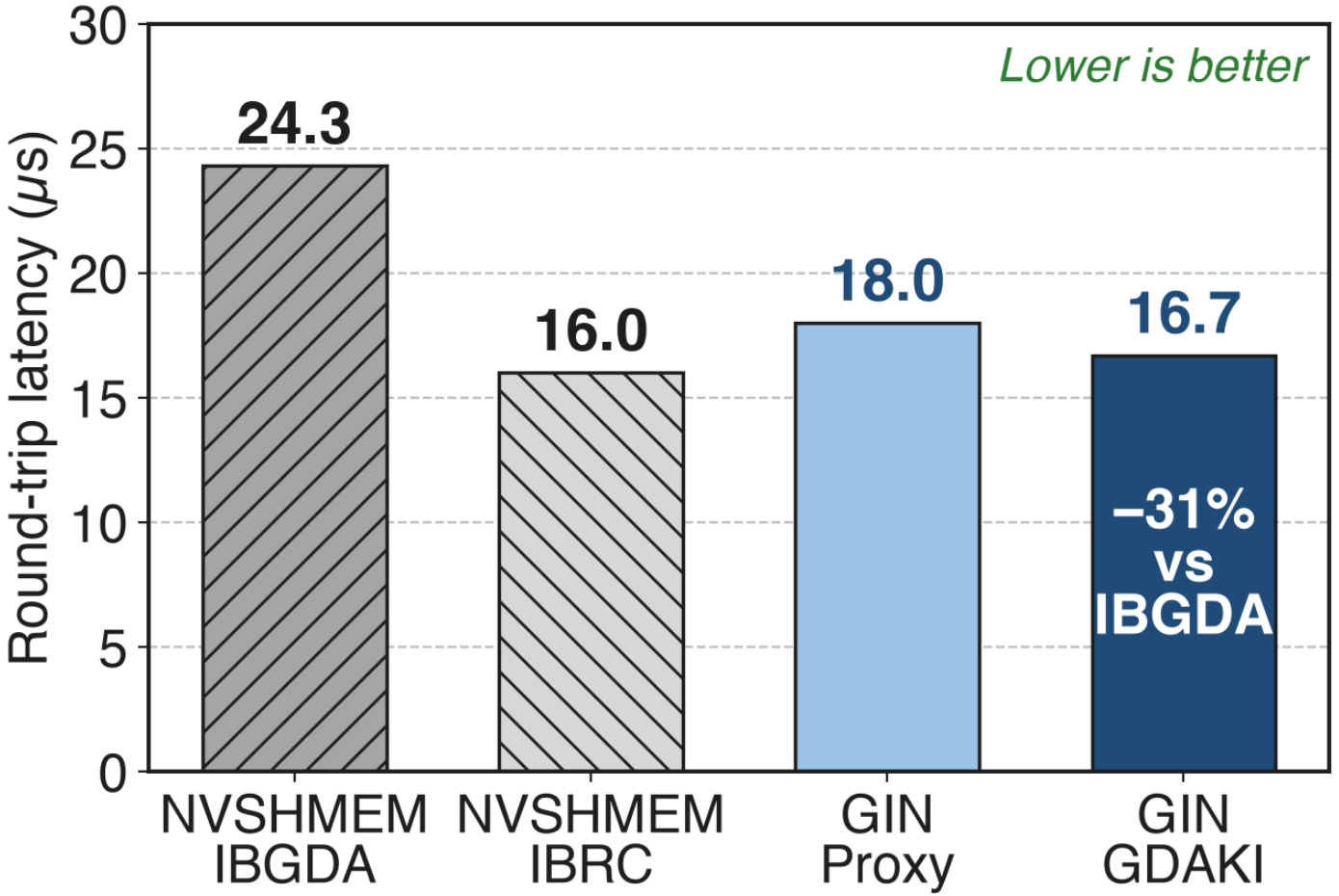
Same context, same peer: the signal lands only after every earlier put is visible

❸ Contexts are QPs; GDAKI is mandatory

- Exclusive context per QP; SMs own disjoint QPs
- Rail links in hybrid mode, full mesh in direct
- Traffic class = InfiniBand service level
- GDAKI: GPU rings the NIC; ConnectX-6 Dx+
- Proxy: a CPU thread posts; V2.5 refuses it
- No other RDMA backend

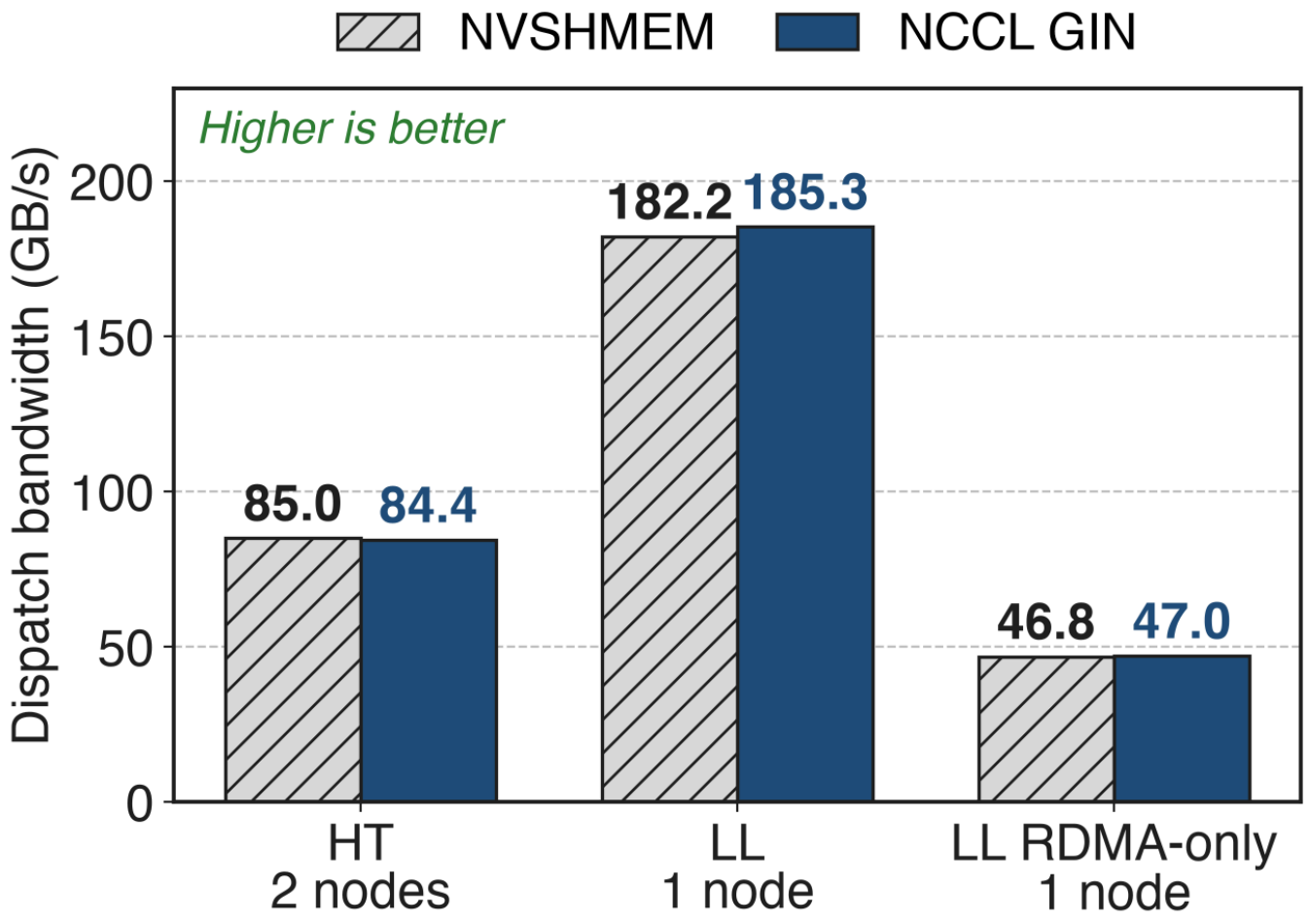
GDAKI's round trip: 16.7 µs vs IBGDA's 24.3

- Put-with-signal ping-pong, 4–128 B
- Two H100s; NCCL 2.28, NVSHMEM 3.4.5
- IBRC's 16.0 µs is on par



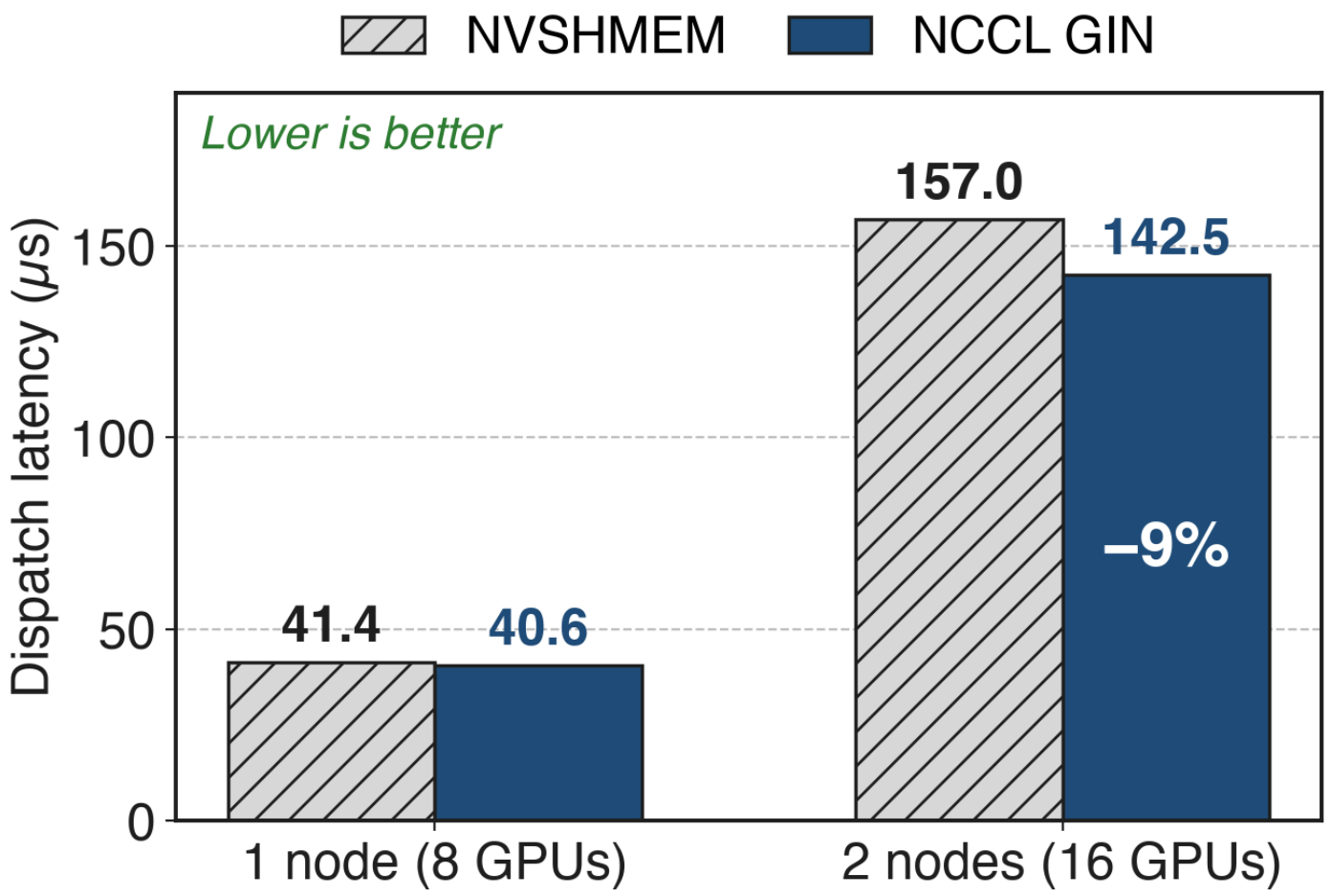
Inside DeepEP, GIN matches NVSHMEM within 2%

- NVIDIA's GIN port of DeepEP v1.2.1
- H100 cluster, BF16, 24 SMs per GPU
- 8 nodes HT: both ≈ 53–54 GB/s



GIN cuts two-node decode dispatch latency 9%

- Low-latency kernels with NVLink
- BF16, hidden 7168, 1–128 tokens
- Pure RDMA at 8 nodes: both 219–225 µs



8 NVSHMEM knobs become zero

8 → 0

NVSHMEM variables: V2 reuses the job's NCCL communicator and V2.5 deletes NVSHMEM; RDMA setup shrinks to a few EP_* flags, e.g. the IB service level

DeepEP README (V2 release b306af0; V2.5 main) and PR #763

Costs: version lockstep and GDAKI-capable NICs

- **NCCL ≥ 2.32.3**, the same library in PyTorch and DeepEP
- **GDAKI only**: ConnectX-6 Dx or newer, GPU and NIC on one PCIe root
- Evidence is a v1.2.1 port on H100; InfiniBand only

Summary

- V1: a second runtime, **8** NVSHMEM knobs, 3 handshakes
- **Insight**: NCCL can hand GPU threads the NIC doorbell
- One communicator, one window; put, signal, get from warps
- **16.7 vs 24.3 µs** round trip; DeepEP parity **within 2%**

https://github.com/deepseek-ai/DeepEP